Pattern Recognition Decision Framework

How to Identify the Right Pattern in 60 Seconds

This guide helps you look at a problem and immediately know which pattern to apply. Look for these **trigger words and characteristics** in the problem statement.

🔍 Step 1: Read the Problem - Look for These Keywords

Input Type Triggers

If you see...	Consider Pattern	Confidence
"**array**" or "**list**" + "**sorted**"	Two Pointers or Binary Search	High
"**array**" + "**pairs**", "**triplets**"	Two Pointers	Very High
"**substring**" or "**subarray**"	Sliding Window	Very High
"**consecutive**" elements	Sliding Window or Greedy	High
"**linked list**" + "**cycle**"	Fast & Slow Pointers	Very High
"**linked list**" + "**reverse**"	Linked List Pattern	Very High
"**tree**" or "**binary tree**"	BFS/DFS	Very High
"**graph**"	BFS/DFS	Very High
"**parentheses**" or "**brackets**"	Stack	Very High
"**intervals**" or "**meetings**"	Intervals/Greedy	Very High
"**matrix**" or "**2D grid**"	DFS/BFS or DP	Medium

Action/Goal Triggers

If you see...	Consider Pattern	Confidence
"**find pair that sums to...**"	Two Pointers or Hash Map	Very High
"**maximum/minimum subarray**"	Sliding Window or DP	High
"**longest substring without...**"	Sliding Window	Very High

If you see...	Consider Pattern	Confidence
"all combinations" or **"all permutations"**	Backtracking	Very High
"minimum/maximum steps"	BFS or DP	High
"can you reach..."	BFS or Greedy	Medium
"count number of ways"	Dynamic Programming	Very High
"optimize" or **"minimize/maximize"**	DP or Greedy	Medium
"valid" or **"balanced"**	Stack	High
"next greater/smaller"	Monotonic Stack	Very High
"search" in **sorted** data	Binary Search	Very High
"kth largest/smallest"	Heap or Quick Select	High

Constraint Triggers

If you see...	Consider Pattern	Why
O(n) time required	Hash Map, Two Pointers, or Sliding Window	Can't use nested loops
O(1) space required	Two Pointers or Bit Manipulation	Can't use extra structures
O(log n) time	Binary Search or Binary Tree	Must eliminate half each time
n ≤ 20	Backtracking or Bitmask DP	Small enough for exponential
n ≤ 1000	$O(n^2)$ or $O(n^2 \log n)$ okay	DP or nested loops fine
n ≤ 10^6	Must be O(n) or O(n log n)	Hash Map, Two Pointers, Sort

🎯 Step 2: Quick Pattern Decision Tree

```
START: Read problem statement
  |
  ├─ Is input SORTED?
  |   ├─ YES → Need to FIND something?
  |   |   ├─ Pairs/Triplets → TWO POINTERS ✓
  |   |   └─ Single element → BINARY SEARCH ✓
  |   └─ NO → Continue...
  |
```

```
├─ Does it mention SUBSTRING or SUBARRAY?
│  ├─ With condition/constraint → SLIDING WINDOW ✓
│  └─ Without condition → Continue...
│
├─ Is it about LINKED LIST?
│  ├─ Mentions cycle/loop → FAST & SLOW POINTERS ✓
│  ├─ Reverse/reorder → LINKED LIST MANIPULATION ✓
│  └─ Continue...
│
├─ Does it say "ALL" combinations/permutations/subsets?
│  └─ YES → BACKTRACKING ✓
│
├─ Is it about TREE or GRAPH?
│  ├─ Level-by-level → BFS ✓
│  ├─ Explore paths → DFS ✓
│  └─ Continue...
│
├─ Mentions PARENTHESES, BRACKETS, or MATCHING?
│  └─ YES → STACK ✓
│
├─ Mentions INTERVALS or MEETINGS?
│  └─ YES → INTERVALS/GREEDY ✓
│
├─ Says "COUNT number of WAYS" or "MINIMUM/MAXIMUM ways"?
│  └─ YES → DYNAMIC PROGRAMMING ✓
│
├─ Mentions FREQUENCY or COUNTING?
│  └─ YES → HASH MAP ✓
│
└─ None of the above?
   └─ Look at constraints and time complexity requirements
```

🚦 Step 3: Pattern Recognition by Problem Characteristics

Characteristic 1: What are you finding?

Finding PAIRS/TRIPLETS

Keywords: "pair", "two numbers", "three numbers", "sum equals"
Pattern: TWO POINTERS (if sorted) or HASH MAP (if unsorted)

Example signals:
- "Find two numbers that sum to target"
- "Find three numbers that sum to zero"
- "Find pair with smallest difference"

Finding SUBARRAY/SUBSTRING

Keywords: "subarray", "substring", "consecutive", "contiguous"
Pattern: SLIDING WINDOW

Example signals:
- "Longest substring without repeating characters"
- "Maximum sum subarray of size k"
- "Minimum window substring"
- "Find all anagrams in a string"

Finding ALL possibilities

Keywords: "all combinations", "all permutations", "all subsets", "generate all"
Pattern: BACKTRACKING

Example signals:
- "Generate all possible..."
- "Find all combinations that..."
- "List all permutations"

Finding MINIMUM/MAXIMUM with choices

Keywords: "minimum cost", "maximum profit", "count ways", "can you reach"
Pattern: DYNAMIC PROGRAMMING or GREEDY

Example signals:
- "What's the minimum cost to..."
- "Count the number of ways to..."
- "Maximum profit from..."
- Use DP if: overlapping subproblems, need to try all options
- Use Greedy if: local optimal = global optimal

Characteristic 2: What's the data structure?

ARRAY (Unsorted)

Common patterns:
1. Need frequency? → HASH MAP
2. Need to find pairs? → HASH MAP or TWO POINTERS (after sort)
3. Need subarray? → SLIDING WINDOW or PREFIX SUM
4. Processing in order matters? → STACK

Quick check:
- Can I sort it? → Consider TWO POINTERS
- Do I need original order? → HASH MAP or SLIDING WINDOW

ARRAY (Sorted)

Common patterns:
1. Search for element? → BINARY SEARCH
2. Find pairs? → TWO POINTERS
3. Find range? → BINARY SEARCH (find start + find end)

Quick check:
- O(n)? → TWO POINTERS
- O(log n)? → BINARY SEARCH

LINKED LIST

Common patterns:
1. Find cycle? → FAST & SLOW POINTERS
2. Reverse? → LINKED LIST MANIPULATION
3. Merge/split? → TWO POINTERS
4. Find middle? → FAST & SLOW POINTERS

Quick check:
- Mentions "cycle" or "loop"? → FAST & SLOW
- Need to modify structure? → POINTER MANIPULATION

TREE/GRAPH

Common patterns:
1. Level-by-level processing? → BFS (use queue)
2. Explore all paths? → DFS (recursive or stack)
3. Find shortest path? → BFS
4. Detect cycle in graph? → DFS with visited set

Quick check:
- "Level order"? → BFS
- "All paths"? → DFS
- "Shortest"? → BFS

STRING

Common patterns:
1. Substring problem? → SLIDING WINDOW
2. Palindrome? → TWO POINTERS or DP
3. Pattern matching? → HASH MAP or DP
4. Parentheses? → STACK

Quick check:
- "Substring" + condition? → SLIDING WINDOW
- "Valid" or "balanced"? → STACK

📊 Step 4: The 5-Second Pattern Quiz

When you see a problem, ask these 5 questions in order:

Question 1: Is it sorted?

- **YES + searching** → Binary Search
- **YES + finding pairs** → Two Pointers
- **NO** → Next question

Question 2: Does it mention "sub" (substring/subarray)?

- **YES** → Sliding Window (95% of the time)
- **NO** → Next question

Question 3: Does it say "all" (combinations/permutations)?

- **YES** → Backtracking
- **NO** → Next question

Question 4: Is it a tree/graph?

- **YES + level/shortest** → BFS
- **YES + paths/explore** → DFS
- **NO** → Next question

Question 5: Does it ask for "ways" or "minimum/maximum"?

- **YES** → Dynamic Programming or Greedy
- **NO** → Default to Hash Map for frequency/lookup problems

💡 Real Examples - Pattern Recognition Practice

Example 1

Problem: "Given an array of integers, find two numbers that add up to a target."

Recognition Process:

1. Array input ✓
2. Finding PAIRS ✓
3. Not sorted (usually)
4. **Pattern: HASH MAP** (or sort first + Two Pointers)

Example 2

Problem: "Find the longest substring without repeating characters."

Recognition Process:

1. STRING input ✓
2. Keyword: "SUBSTRING" ✓
3. With condition: "without repeating"
4. **Pattern: SLIDING WINDOW**

Example 3

Problem: "Given a sorted array, find the first and last position of target."

Recognition Process:

1. Array is SORTED ✓

2. SEARCHING for position
3. Need O(log n) for optimal
4. **Pattern: BINARY SEARCH** (run twice)

Example 4

Problem: "Determine if a linked list has a cycle."

Recognition Process:

1. LINKED LIST ✓
2. Keyword: "CYCLE" ✓
3. **Pattern: FAST & SLOW POINTERS**

Example 5

Problem: "Generate all possible subsets of a set."

Recognition Process:

1. Keyword: "ALL" ✓
2. Keyword: "subsets" = combinations
3. **Pattern: BACKTRACKING**

Example 6

Problem: "Find minimum cost to climb stairs (can climb 1 or 2 steps)."

Recognition Process:

1. Keywords: "MINIMUM cost" ✓
2. Has CHOICES (1 or 2 steps)
3. Overlapping subproblems
4. **Pattern: DYNAMIC PROGRAMMING**

Example 7

Problem: "Check if parentheses are balanced."

Recognition Process:

1. STRING input
2. Keyword: "PARENTHESES" ✓
3. Need to MATCH pairs
4. **Pattern: STACK**

Example 8

Problem: "Merge overlapping intervals."

Recognition Process:

1. Keyword: "INTERVALS" ✓
2. Need to merge/combine
3. **Pattern: INTERVALS** (sort + merge)

🎓 Pattern Recognition Training Exercises

Practice identifying patterns WITHOUT solving. Just read and identify:

1. "Find kth largest element in array" → **Heap or Quick Select**
2. "Longest increasing subsequence" → **Dynamic Programming**
3. "Valid binary search tree" → **DFS (Tree)**
4. "Coin change: minimum coins to make amount" → **Dynamic Programming**
5. "Next greater element for each element" → **Monotonic Stack**
6. "Search in rotated sorted array" → **Modified Binary Search**
7. "Word ladder (shortest transformation)" → **BFS**
8. "House robber (can't rob adjacent)" → **Dynamic Programming**
9. "Remove nth node from end of list" → **Two Pointers (Linked List)**
10. "Find duplicate number in array" → **Fast & Slow Pointers or Hash Map**

🔥 The Ultimate Pattern Identification Checklist

Print this and keep it next to you while practicing:

Before writing ANY code, check these in order:

- [ ] Is the input sorted? (Binary Search or Two Pointers likely)
- [ ] Does it mention "substring" or "subarray"? (Sliding Window likely)
- [ ] Does it say "all" combinations/permutations? (Backtracking)
- [ ] Is it about linked list + cycle? (Fast & Slow Pointers)
- [ ] Is it about tree/graph traversal? (BFS or DFS)
- [ ] Does it mention parentheses/brackets? (Stack)
- [ ] Does it mention intervals/meetings? (Intervals pattern)
- [ ] Does it ask for "ways" or "minimum/maximum"? (DP or Greedy)

- [ ] Do I need to find pairs/triplets? (Two Pointers or Hash Map)
- [ ] Do I need frequency counting? (Hash Map)

If still unsure:

- [ ] What's the time complexity requirement?
- [ ] What's the space complexity requirement?
- [ ] What's the constraint on n?

🎯 Pro Tips for Faster Recognition

1. **First 10 seconds:** Read problem and identify data structure
2. **Next 10 seconds:** Look for trigger keywords
3. **Next 10 seconds:** Check constraints
4. **Next 30 seconds:** Decide pattern and verify it makes sense
5. **Start coding:** You should know the pattern within 60 seconds

Common mistakes to avoid:

- Don't overthink - if you see "substring", it's probably Sliding Window
- Don't ignore "sorted" - it's a huge hint
- Don't miss words like "all", "cycle", "intervals" - they're dead giveaways
- Trust the keywords more than your intuition at first

Master this skill and you'll save 5-10 minutes per problem in actual tests!